DoT Front-end

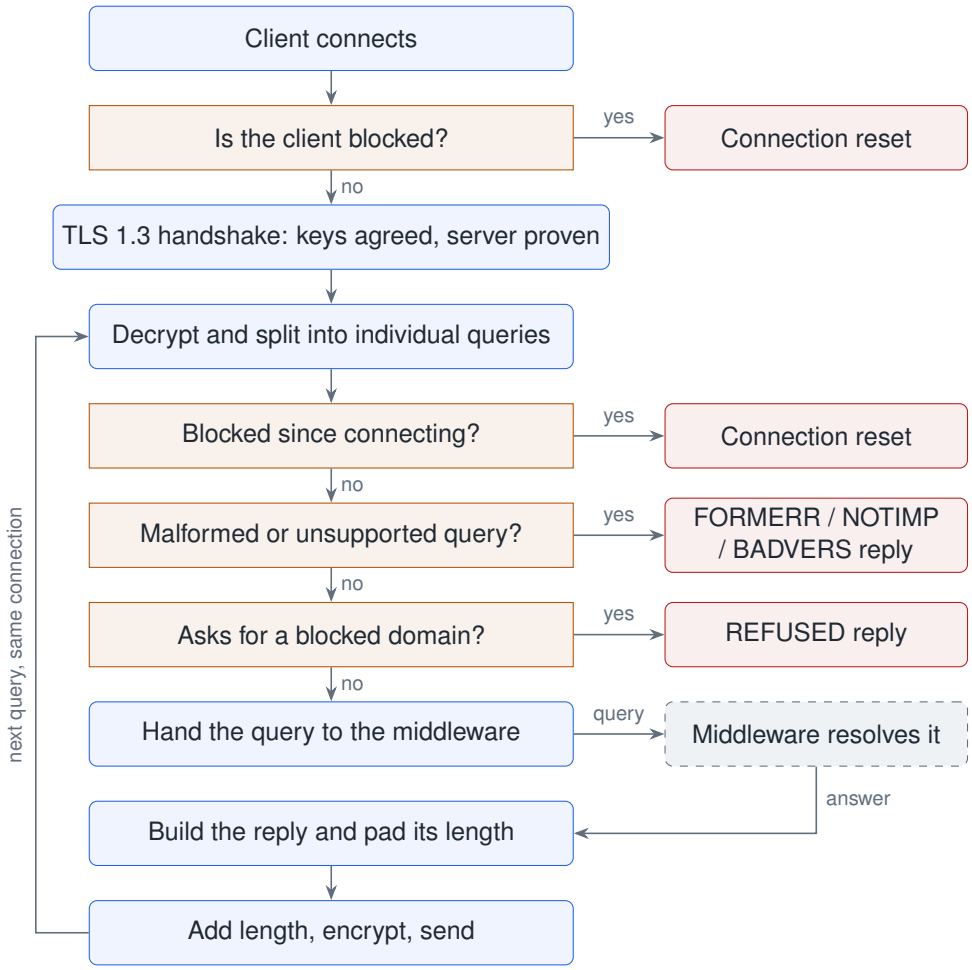
Design and progress

Owner: Dhruv · 28 October 2026

The front-end is the resolver's front door. It accepts encrypted DNS queries from clients over DNS-over-TLS (TCP port 853), checks each one, passes valid queries to the middleware, and sends back the encrypted answers. It also decides which clients and queries get rejected, and how.

All 11 features are complete and tested, including end to end with `dig +tls`.

How a query flows through the front-end



A connection stays open after a reply, so one client can send many queries without repeating the TLS handshake. It may send several without waiting (pipelining); each answer goes out as soon as it is ready, matched to its query by ID.

Features

Feature	What it does	Status
Query parsing and validation	Turns raw bytes into a checked query; rejects anything malformed or hostile without ever reading outside the message	**Done**
Name checks	Accepts names made of letters, digits, - and _; compares them without regard to case, but echoes the client's own spelling back	**Done**
Reply building	Turns the middleware's answer (answer, authority and additional records) into a correct DNS reply, always producing a valid message (falls back to SERVFAIL if needed)	**Done**
Length padding	Pads replies to a fixed block size (468 bytes) so encrypted traffic doesn't reveal which site was looked up by its size	**Done**
Message framing	Splits the decrypted byte stream into individual queries, and prefixes each reply with its length	**Done**
TLS encryption	Encrypts every connection with OpenSSL, TLS 1.3 only; proves the server's identity with its certificate	**Done**
Abuse blocking	Counts misbehaviour per client; a client that misbehaves too often is cut off before any TLS work, for a set time	**Done**
Policy rejections	Answers REFUSED for queries about blocked domains, read from a file	**Done**
Connection handling	Serves many clients at once, several queries per connection, with timeouts and limits	**Done**
Configuration	Listen address, certificate and key, limits, blocking settings, blocked-domain list	**Done**
Runnable server	`dns_server <config-file>`, tested end to end with `dig +tls`	**Done**

How the front-end rejects queries

Situation	Response	Why
Client is blocked for abuse	Connection reset, no reply, no TLS work	No work or bandwidth spent on attackers
Message too broken to read its ID, or not a query at all	Connection closed, no reply	There is nothing valid to reply to
Malformed query	FORMERR	Tells a buggy client what's wrong
Query kind we don't support	NOTIMP	A clear answer instead of silence
EDNS version we don't support	BADVERS	The client retries with version 0, keeping EDNS and padding
Query asks for a blocked domain	REFUSED, connection stays open	A clear answer; the client isn't punished
Middleware overloaded, or client over its rate limit	SERVFAIL / REFUSED	The client can retry later

What the response codes mean

Every DNS reply carries a response code. FORMERR, NOTIMP, BADVERS and REFUSED say the problem is with the query; SERVFAIL says the problem is on the resolver's side, so retrying later can succeed.

Code	What it means in this resolver	Retry?
FORMERR Format error · code 1	**The query is broken.** It was readable enough to reply to, but breaks DNS rules: wrong section counts, a damaged name or one with characters outside letters, digits, − and _, a bad EDNS record, or extra bytes at the end	No
NOTIMP Not implemented · code 4	**Valid, but unsupported.** An operation other than a standard query, a class other than Internet (e.g. CHAOS), or a record type outside A, NS, CNAME, SOA, PTR, MX, TXT, AAAA and ANY (e.g. HTTPS)	No
BADVERS Bad EDNS version · code 16	**Wrong EDNS version.** The client used a version above 0, the only one that exists; the reply says "use version 0"	Yes, with v0
REFUSED Refused · code 5	**Understood, but not allowed.** The domain is on the blocked list, or the client is over the middleware's rate limit	No
SERVFAIL Server failure · code 2	**No answer could be produced.** The query was fine, but the middleware was overloaded, the upstream servers failed, or the answer couldn't be turned into a valid reply	Yes, later

Abuse blocking

Each client earns **strikes** for misbehaving. Too many strikes within a time window and its address is blocked: every new connection from it is reset before the TLS handshake, and an open connection is reset at its next query. Blocks expire on their own.

Counts as a strike	Doesn't count
Being refused by the rate limiter (*rate-limit strike*)	Asking for a blocked domain
A failed TLS handshake (*protocol strike*)	An unsupported query (NOTIMP) or EDNS version (BADVERS)
A zero-length message, one with no readable ID, or a reply instead of a query (*protocol strike*)	Going quiet until the idle timeout
A malformed query, FORMERR (*protocol strike*)	

Protocol strikes have their own, **higher** limit, because honest clients occasionally fail a handshake or send something odd. IPv6 clients are counted per /64 network, since one device usually owns a whole /64. A client reaching us as `::ffff:1.2.3.4` is the same client as `1.2.3.4`.

Limits that protect the server

Limit	Default	What happens
Connections at once	1000	Extra connections are closed straight away
Idle connection	10 s	Closed; also covers a client that never finishes its handshake. A connection still waiting for an answer is never idle
Queries waiting per connection	100	The server stops reading from that client until answers go out
Unsent answers per connection	256 KiB	A client that stops reading its answers is closed
Clients tracked for strikes	100,000	When full, new clients aren't tracked; existing blocks always stay

How the encryption works

- **TLS 1.3 only.** Every TLS 1.3 cipher suite uses an ephemeral key exchange and authenticated encryption; older versions are refused.
- **The session key is never sent.** Client and server each compute it from a key exchange (ECDHE), so someone watching the network can't obtain it.
- **The server proves its identity** by signing the handshake with its private key. The key never leaves the server, and the signature stops anyone impersonating it.
- **Forward secrecy.** A stolen private key would let an attacker impersonate the server in future, but not decrypt traffic recorded earlier.
- **Length padding** covers the one thing TLS doesn't hide: message sizes.
- **Key file protection.** The server warns at startup if the private key can be read by other users, and `*.pem` / `*.key` files are kept out of git.
- **ALPN.** A client offering the `dot` protocol name gets it confirmed; it isn't required.

What the front-end uses from the middleware

The middleware is built by a teammate; the front-end calls three of its functions:

- `submit(name, type, client, callback)` hands over a query. The callback runs exactly once with the answer, possibly on another thread; the front-end passes the encoded reply back to its own I/O thread.
- `is_blocked(address)` answers "is this client blocked right now?", at connect time and before each query.
- `report_abuse(address)` records a protocol strike.

Configuration

These keys go in the same config file as the middleware's (see `config/middleware.example.conf`):

Key	Default	Meaning
`listen`	`0.0.0.0:853`	Address and port; `[::]:853` also accepts IPv4
`tls_cert`, `tls_key`	none	Certificate chain and private key (PEM). Both are required to run the server
`max_connections`	1000	Connections at once
`idle_timeout_ms`	10000	Idle time before a connection is closed
`blocked_domains_file`	none	One domain per line, `#` comments allowed. A domain also blocks everything under it, so a lone `.` refuses every query
`block_rate_limit_strikes`	0 (off)	Rate-limit strikes before a block
`block_protocol_strikes`	0 (off)	Protocol strikes before a block; must be higher than the rate-limit one
`block_window_ms`	10000	How long a strike is remembered
`block_duration_ms`	300000	How long a block lasts
`block_max_entries`	100000	Clients tracked for strikes

Building, running and testing

Needs Linux, `g++-14`, CMake and OpenSSL 3 (`sudo apt install g++-14 libssl-dev`). On other systems everything except the DoT server itself still builds.

```
cmake --preset local && cmake --build build/local    # build
ctest --test-dir build/local                         # run all tests

# a self-signed certificate for local testing
openssl req -x509 -newkey ec -pkeyopt ec_paramgen_curve:P-256 -nodes \
        -keyout server.key -out server.pem -days 30 -subj /CN=localhost
chmod 600 server.key

build/local/dns_server my.conf                       # with tls_cert/tls_key set
dig @127.0.0.1 -p 853 +tls example.com               # query it
```